

ES -116

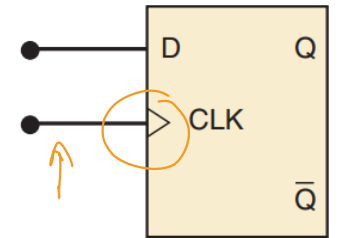
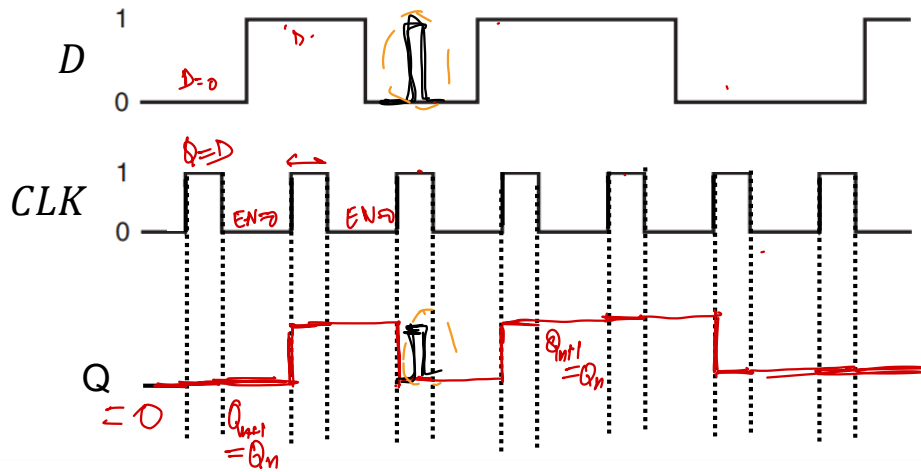
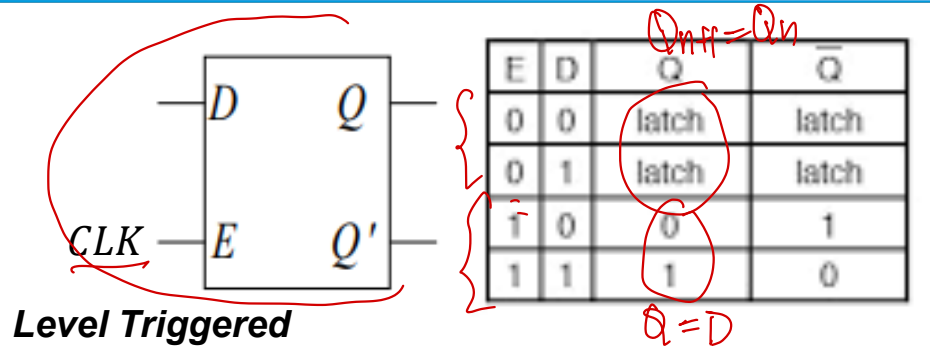
Principles and Applications of Electrical Engineering

Semester II, 2025-26

Review of Sequential Circuits

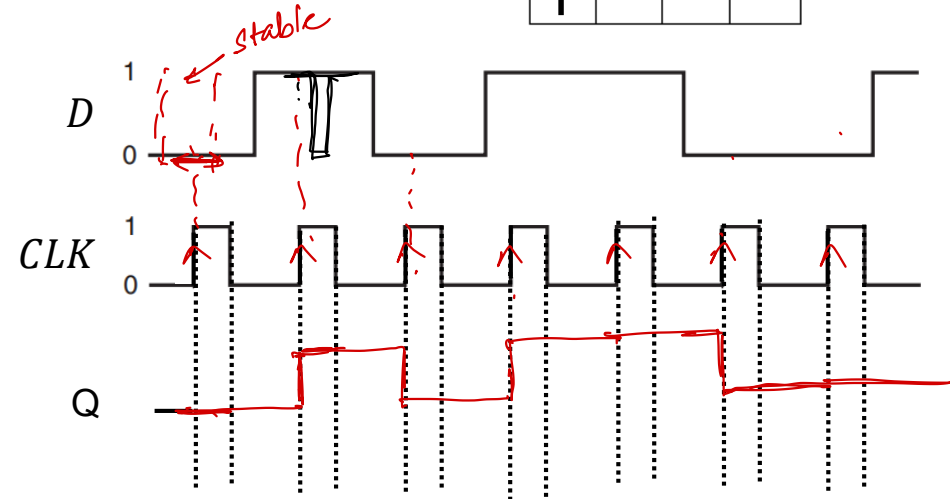
Reference book: Digital Systems Book by Tocci

Latch vs FF

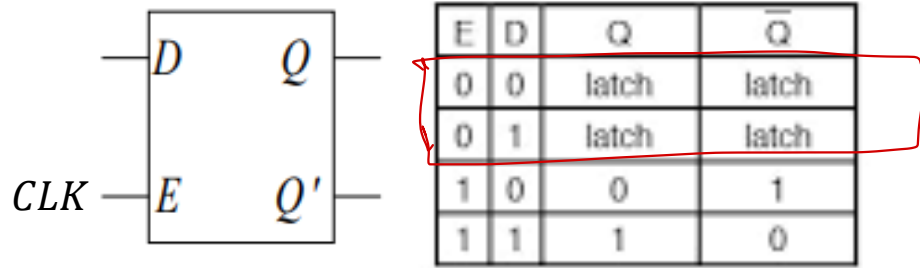


Edge Triggered

clk	D	Q	$\bar{Q}$
	0	Q	$\bar{Q}$
	1	Q	$\bar{Q}$
↑	0	0	1
↑	1	1	0

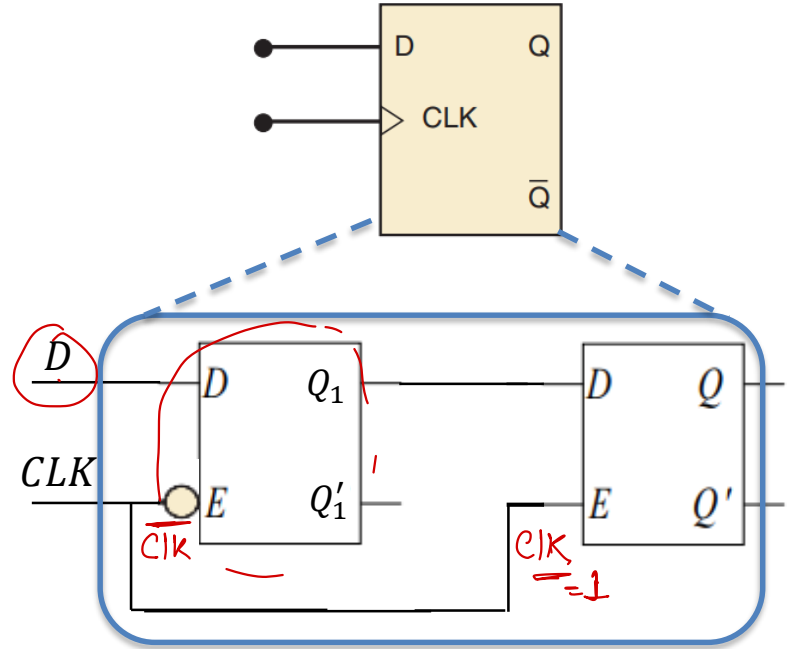
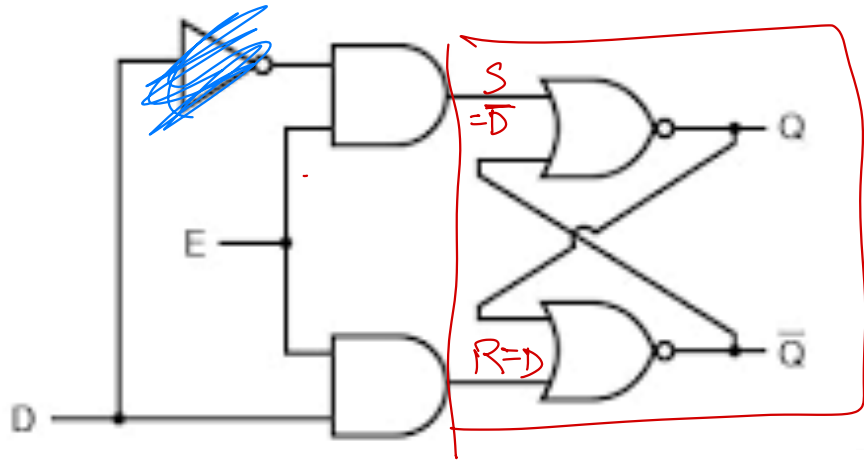


Latch vs FF (differences in implementation)



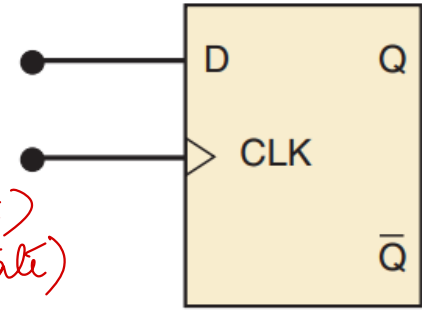
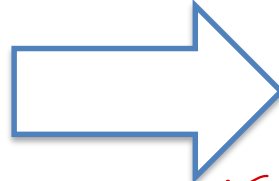
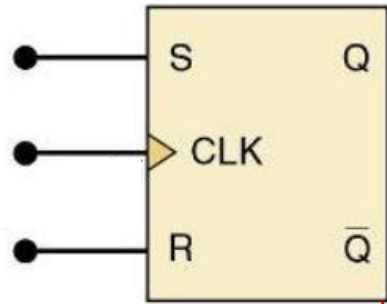
$S=0, R=0, Q_{n+1}=Q_n$

$D=0$



clk	D	Q	$\bar{Q}$
	0	Q	$\bar{Q}$
	1	Q	$\bar{Q}$
↑	0	0	1
↑	1	1	0

Method to convert from SR FF to D FF (Q3)



*D (External Input)
Q_n (Present state)*

Inputs Truth Table Output

S	R	CLK	Q
0	0	↑	Q_0 (no change) = $Q_{n+1} = Q_n = 0$
1	0	↑	1
0	1	↑	0
1	1	↑	Ambiguous

↓ Excitation Table

Q_n	Q_{n+1}	S	R
0	0	0	0
0	1	1	0
1	0	0	1
1	1	0	0

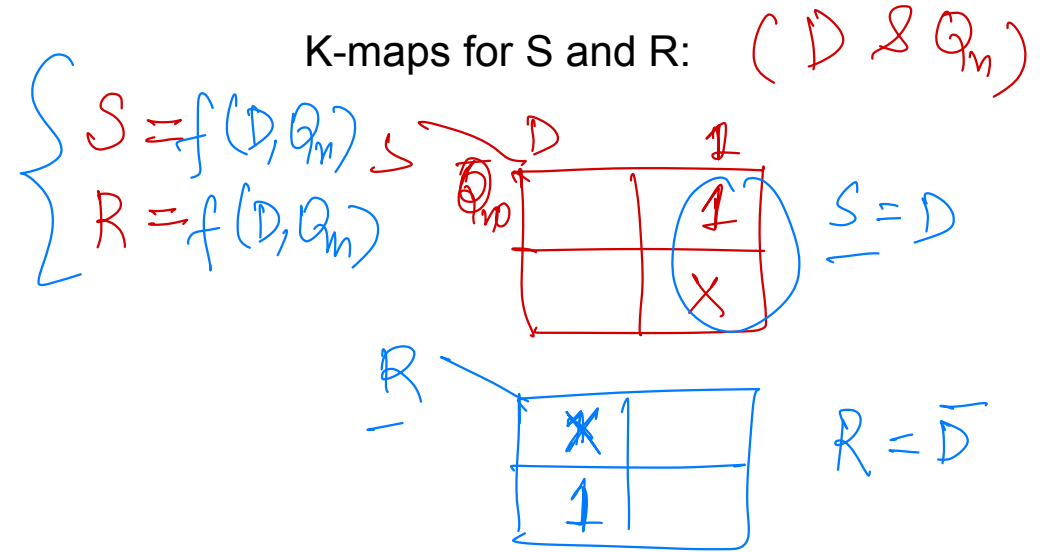
State Table

Input	Present state	Next state	Flip-flop inputs	
D	Q_n	$Q_{n+1} = D$	S	R
0	0	0	0	X
0	1	0	0	1
1	0	1	1	0
1	1	1	X	0

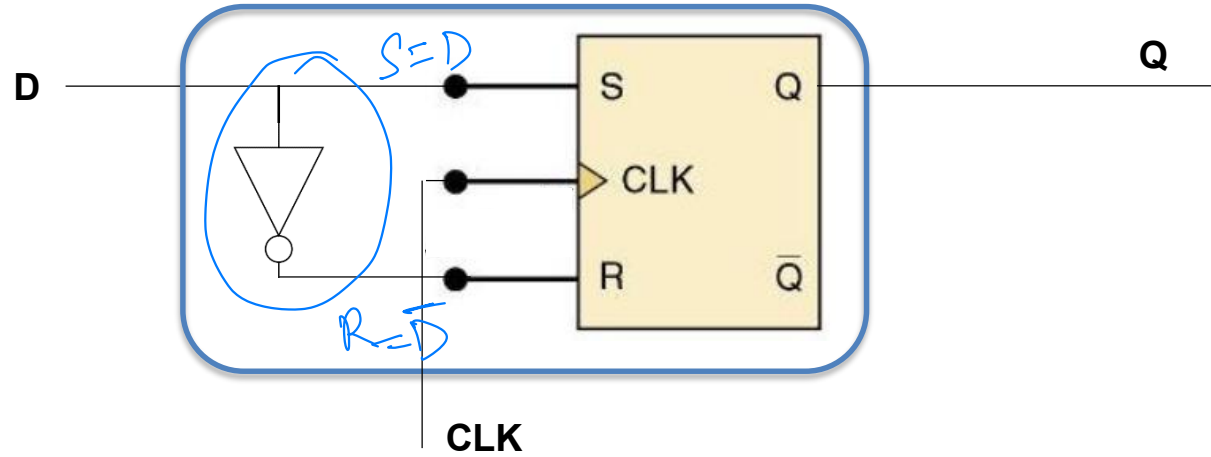
Excitation Table

Write S and R input in terms of D and Q_n

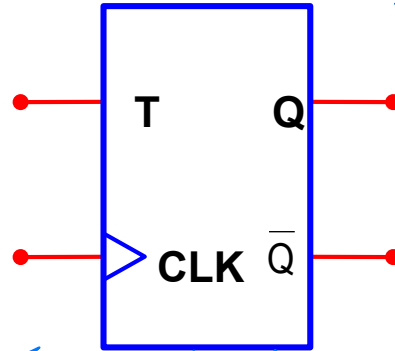
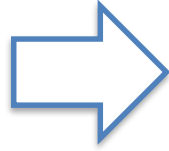
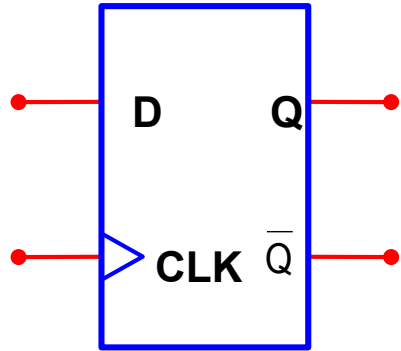
Input	Present state	Next state	Flip-flop inputs	
D	Q_n	Q_{n+1}	S	R
0	0	0	0	X
0	1	0	0	1
1	0	1	1	0
1	1	1	X	0



Logic Circuit for SR to D FF conversion



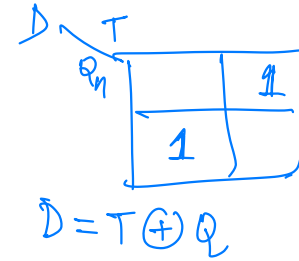
Home work Problem: Building T FF from D FF



$$T=0 \rightarrow Q_{n+1} = Q_n$$

$$T=1 \rightarrow Q_{n+1} = \overline{Q_n}$$

$$D = f(T, Q_n)$$



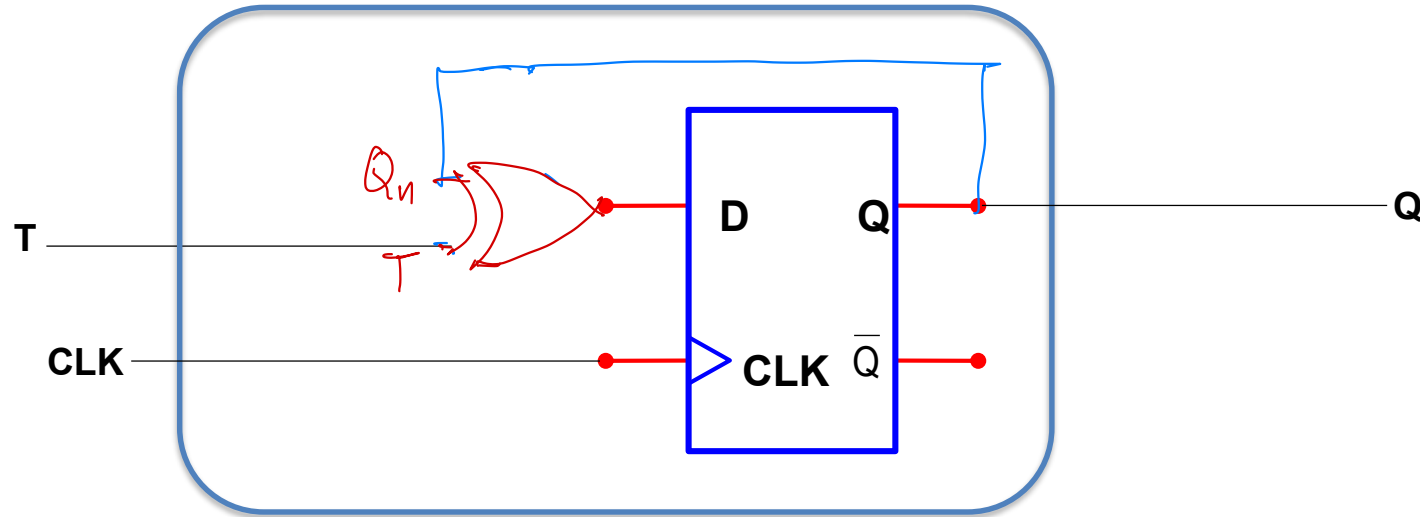
D	Q_n	Q_{n+1}
0	0	0
0	1	0
1	0	1
1	1	1

State Table

T	Q_n	Q_{n+1}	$D = Q_{n+1}$
0	0	0	0
0	1	1	1
1	0	1	1
1	1	0	0

Excitation Table

Logic circuit for T FF from D FF



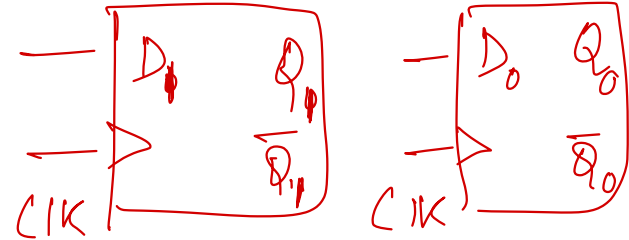
Putting DFF to implement a state diagram (2-bit counter: Q5)

00 → 01 → 10 → 11 → 00

Inputs

State Table

Present State Q_1	Present State Q_0	Next State $Q_1(t+1)$	Next State $Q_0(t+1)$
0	0	0	1
0	1	1	0
1	0	1	1
1	1	0	0



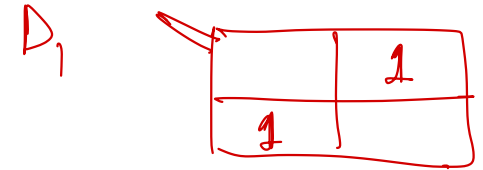
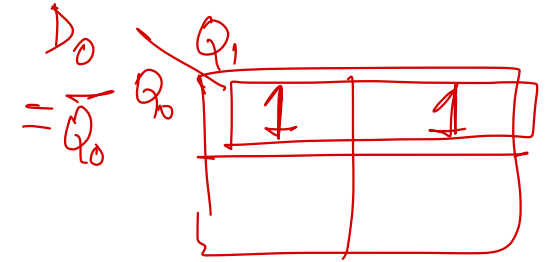
Calculating DFF Inputs to implement this state table

00 $\rightarrow$ 01 $\rightarrow$ 10 $\rightarrow$ 11 $\rightarrow$ 00

Required DFF inputs to make it happen!

Present State Q_1	Present State Q_0	Next State $Q_1(t+1)$	Next State $Q_0(t+1)$	D_1	D_0
0	0	0	1	0	1
0	1	1	0	1	0
1	0	1	1	1	1
1	1	0	0	0	0

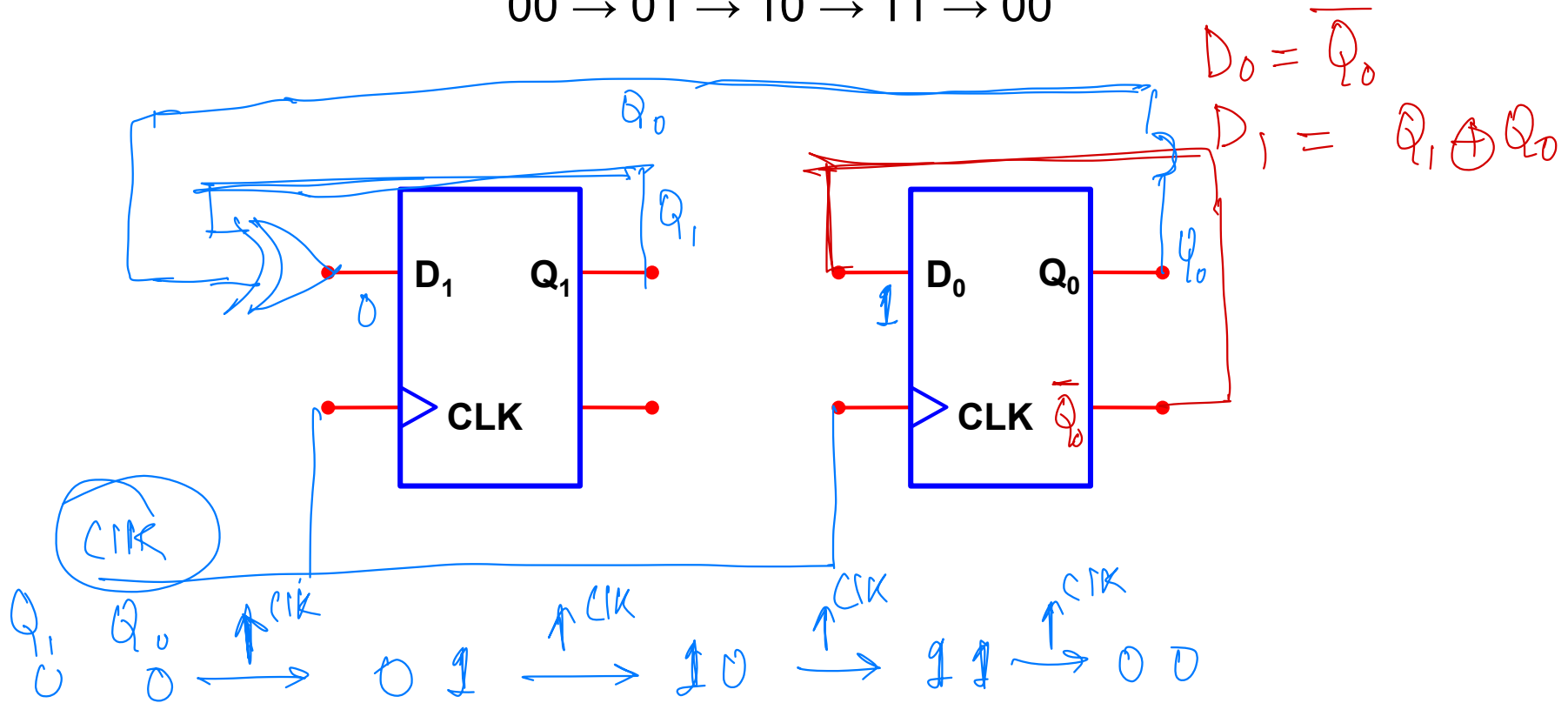
K-maps
(D_0 , D_1 as a function of Q_1 and Q_0)



$$D_1 = Q_1 \oplus Q_0$$

Logic Circuit for 2-bit counter using DFF

00 → 01 → 10 → 11 → 00



Building a state diagram from User Specifications

Q6 (Tutorial): The campus has installed a new automated Chai dispenser. A cup of chai costs exactly 20 Rupees. The machine only accepts 10-Rupee coins.

To save hardware costs, your institute decides to install an "instant" dispenser. The exact moment the second 10-Rupee coin is inserted, it immediately pours the chai and resets. It never "waits" with 20 Rupees inside.

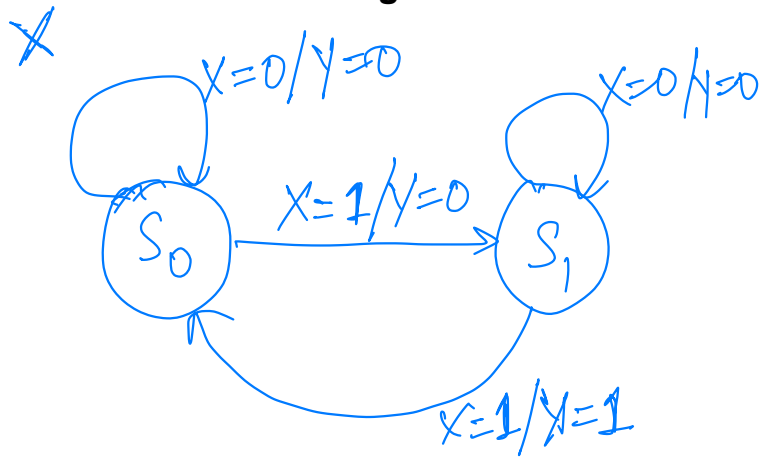
Input X: $X=1$ for exactly one clock cycle when a 10-Rupee coin is inserted ($X=0$ otherwise).

Output Y: $Y=1$ triggers the dispenser to pour the chai ($Y=0$ keeps it locked).

State 0 ($Q=0$) means 0 Rupees inserted. State 1 ($Q=1$) means 10 Rupee inserted.

$Y = (\text{action output})$
1 DFF

State Diagram



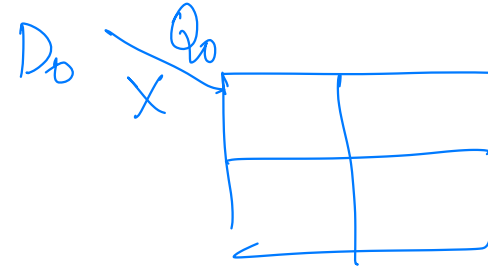
State Table

X	Q_0	$Q_0(t+1)$	$\bar{D}_0$	Y
0	0	0	0	0
0	1	1	1	0
1	0	1	1	0
1	1	0	0	1

State table $\rightarrow$ Input Excitation Table $\rightarrow$ K-maps

X	Present State Q_0	Next State $Q_0(t+1)$	D_0	Y
0	0	0	0	0
1	0	1	1	0
0	1	1	1	0
1	1	0	0	1

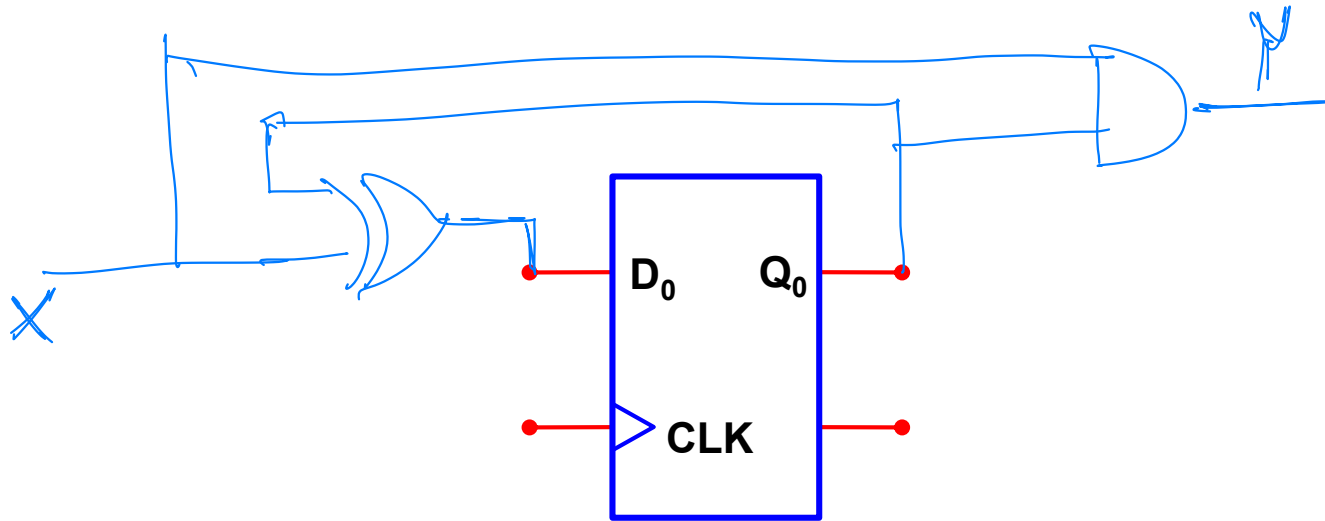
K-maps
(D_0 , Y as a function of Input X and Q_0)



$$D_0 = Q_0 \oplus X$$

$$Y = Q_0 \cdot X$$

Logic circuit to implement this state machine with DFF



Take Home: Building a state diagram from User Specifications

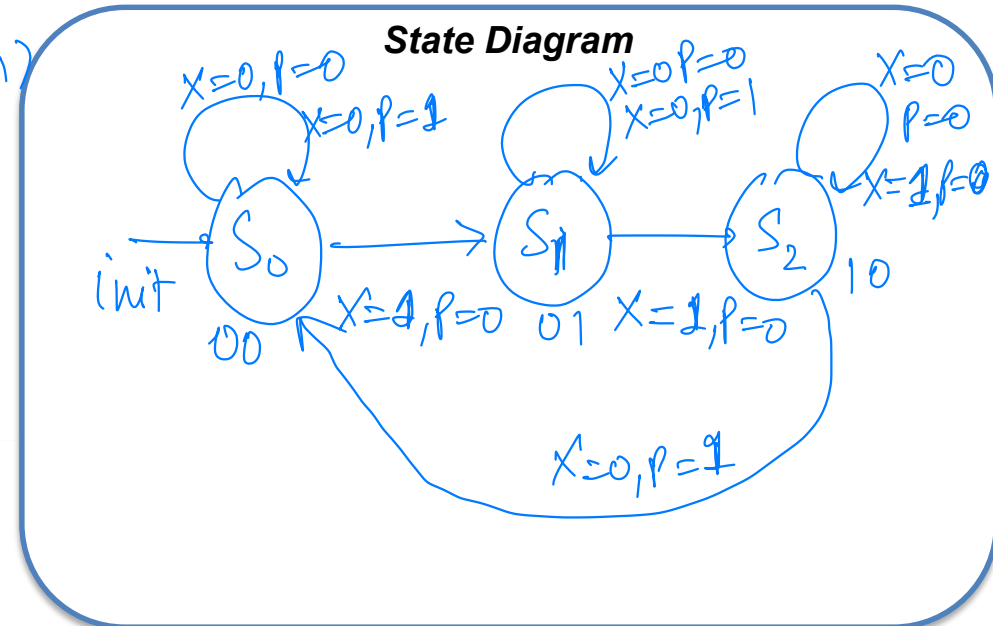
Q6 (Tutorial): The campus has installed a new automated Chai dispenser. A cup of chai costs exactly 20 Rupees. The machine only accepts 10-Rupee coins.

Students complained that the "instant" dispenser from Part A spilled chai on their hands because it poured too fast! You are tasked with upgrading the machine. The machine must now hold the 20 Rupees and wait for the user to press a button before pouring. Input $X=1$ when a 10-Rupee coin is inserted, $P=1$ when the "Pour" button is pressed. (Assume X and P are never pressed at the exact same time). Output Y : $Y=1$ pours the chai. The machine now requires three states: S_0 (00): 0 Rupees, S_1 (01): 10 Rupee, S_2 (10): 20 Rupees (Ready to pour), and S_3 (11): Invalid / Don't Care.

state $\rightarrow X, P, Q_1, Q_0$
table input $\rightarrow$ output $(Q_i(t+1))$

Draw the new State Transition Diagram. Include states S_0 , S_1 , and S_2 . Show what happens if X is inserted, and what happens if P is pressed. (Hint: If P is pressed while in S_0 or S_1 , the machine should just stay in that state).

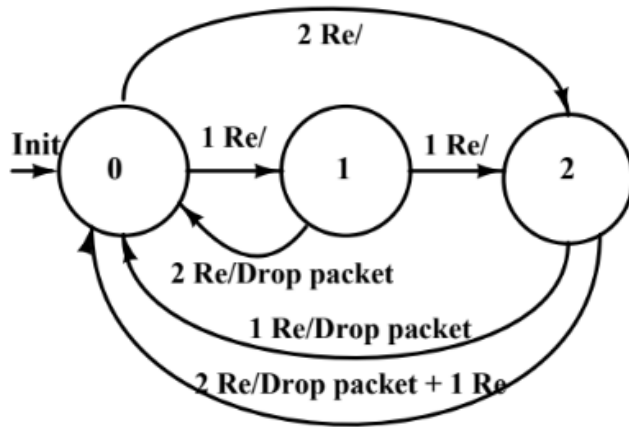
$Q_0(t+1)$



More practice: Penny in the slot machine

Consider a Penny in the slot machine which accepts 1 Re and 2 Re coins. It dispenses a packet worth 3 Rs. and returns coins if required.

State Diagram



Summary as a Table

Curr st	Input	Output	Next St	Input	Output	Next St
0	1 Re	Nothing	1	2 Re	Nothing	2
1	1 Re	Nothing	2	2 Re	Drop packet (P)	0
2	1 Re	Drop packet (P)	0	2 Re	Drop packet (P) + 1 Re (C)	0

State Table

Q_1	Q_0	In_1	In_2	D_1	D_0	P	C

Assume that input “ In_1 ” is TRUE when a 1 Re coin is inserted, while “ In_2 ” is TRUE when a 2 Rs coin is inserted.

Notice that both In_1 and In_2 will never be TRUE at the same time.

State Table (FF input in terms of External Input and Present States)

Consider a Penny in the slot machine which accepts 1 Re and 2 Re coins. It dispenses a packet worth 3 Rs. and returns coins if required.

State Table:

Q1	Q0	In1	In2	D1	D0
0	0	0	0	0	0
0	0	0	1	0	1
0	0	1	0	1	0
0	0	1	1	X	X
0	1	0	0	0	1
0	1	0	1	1	0
0	1	1	0	0	0
0	1	1	1	X	X

State Table:

Q1	Q0	In1	In2	D1	D0
1	0	0	0	1	0
1	0	0	1	0	0
1	0	1	0	0	0
1	0	1	1	X	X
1	1	0	0	X	X
1	1	0	1	X	X
1	1	1	0	X	X
1	1	1	1	X	X

Simplifying using Karnaugh maps gives $D1 = In0 \ Q0 + In1 \ \overline{Q1} \ \overline{Q0} + Q1 \ \overline{In1} \ \overline{In0}$,
 $D0 = \overline{Q1} \ \overline{Q0} \ In0 + Q0 \ \overline{In1} \ \overline{In0}$

Action Table (Drop Packet and/or Drop Coin)

Consider a Penny in the slot machine which accepts 1 Re and 2 Re coins. It dispenses a packet worth 3 Rs. and returns coins if required.

Action Table:

Q1	Q0	In1	In2	P	C
0	0	0	0	0	0
0	0	0	1	0	0
0	0	1	0	0	0
0	0	1	1	X	X
0	1	0	0	0	0
0	1	0	1	0	0
0	1	1	0	1	0
0	1	1	1	X	X

Action Table:

Q1	Q0	In1	In2	P	C
1	0	0	0	0	0
1	0	0	1	1	0
1	0	1	0	1	1
1	0	1	1	X	X
1	1	0	0	X	X
1	1	0	1	X	X
1	1	1	0	X	X
1	1	1	1	X	X

Simplifying using Karnaugh maps gives

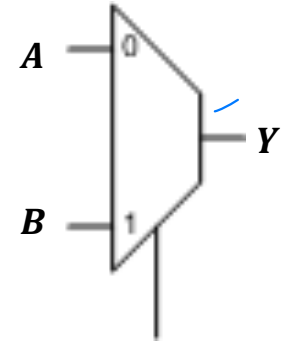
(Drop Packet:) $P = (Q0 + Q1) \cdot In1 + Q1 \cdot In0$

(Drop 1 Re:) $C = Q1 \cdot In1$

Some Surprises: MUX as a logic gate!

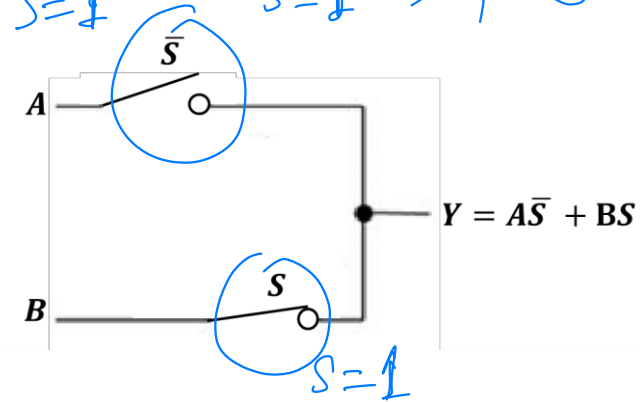
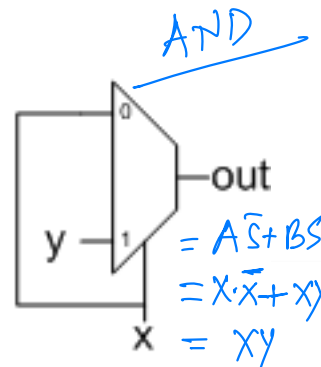
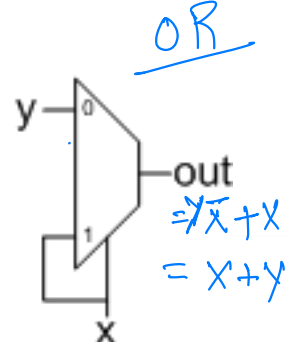
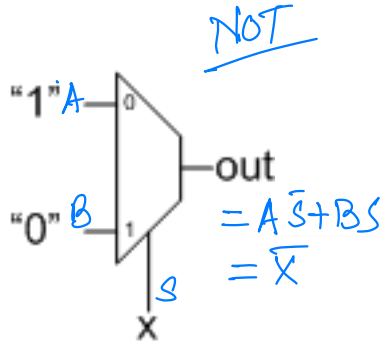
Why bother with MUX based implementation of logic functions?

- A multiplexer can be implemented quite easily in CMOS technology, using two switches controlled by complementary inputs (S and $\bar{S}$).
- Implementation of some hard to optimize logic functions (such as the XOT) is relatively simple using MUX logic.
- Switches can be implemented as single transistors.

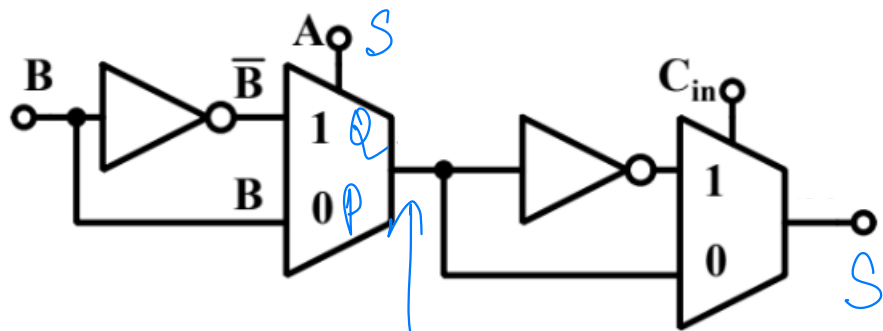


$$\begin{aligned} S=0 &\rightarrow Y=A \\ S=1 &\Rightarrow Y=B \end{aligned}$$

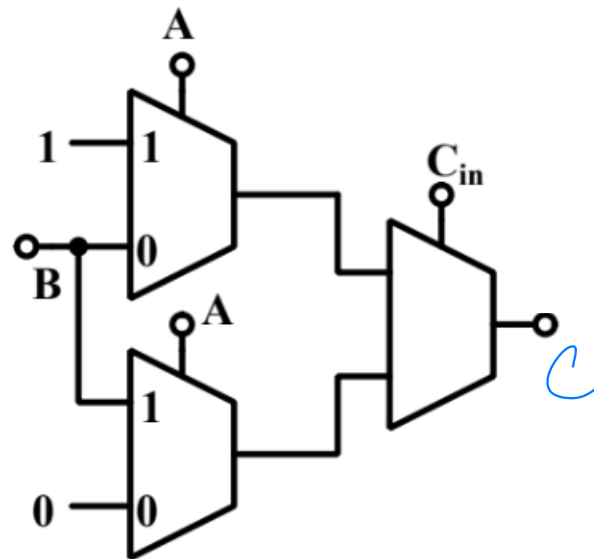
Guess the gates?



More Examples

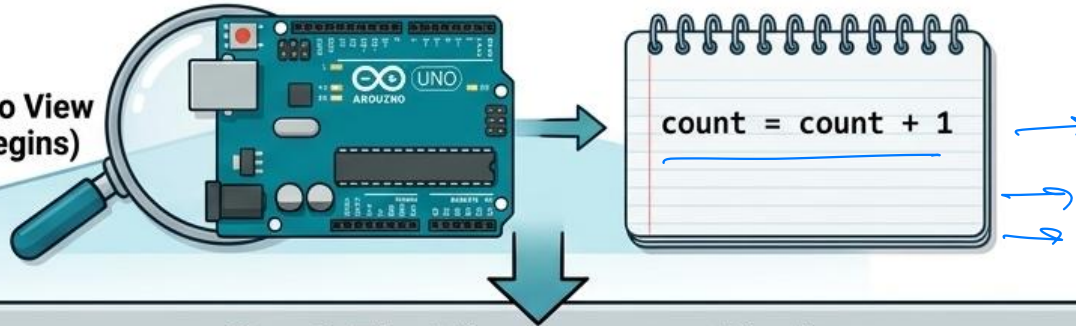


$$\begin{aligned}
 Y &= P\bar{S} + QS \\
 &= \bar{B}\bar{A} + \bar{B}A \\
 &= A \oplus B
 \end{aligned}$$



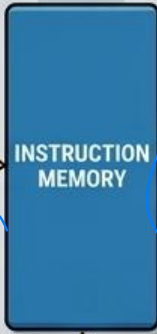
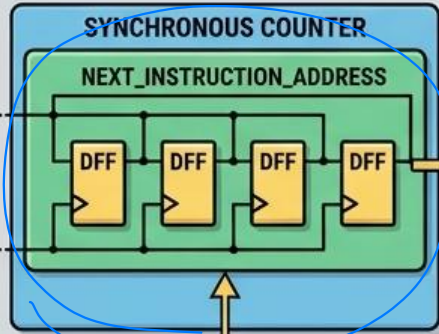
Let's tie it together!

Step 1: The Macro View
(The Zoom-in Begins)

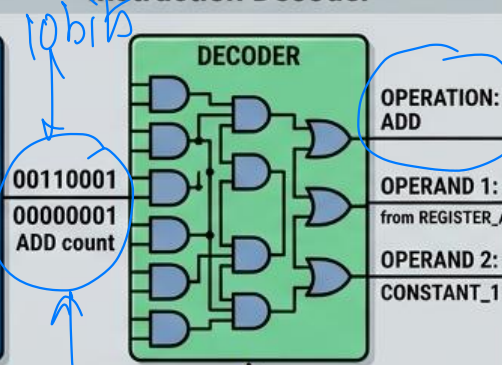


Step 2: The Microprocessor Pipeline

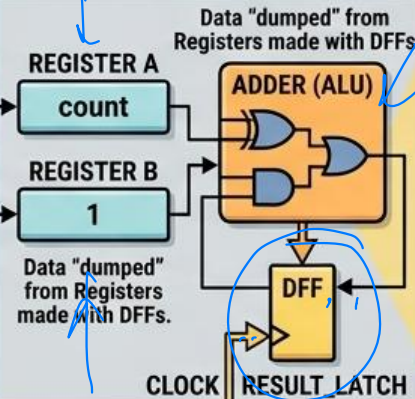
Program Counter (PC)



Instruction Decoder

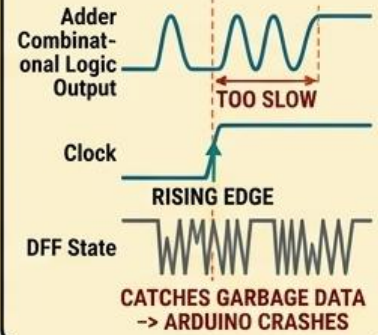


ALU & Registers Block

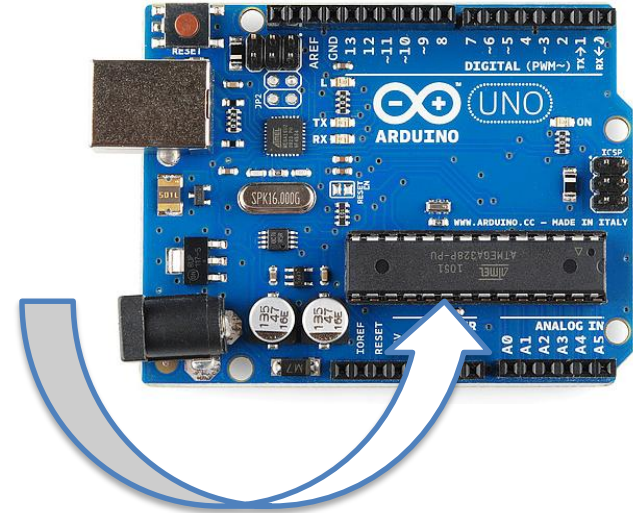
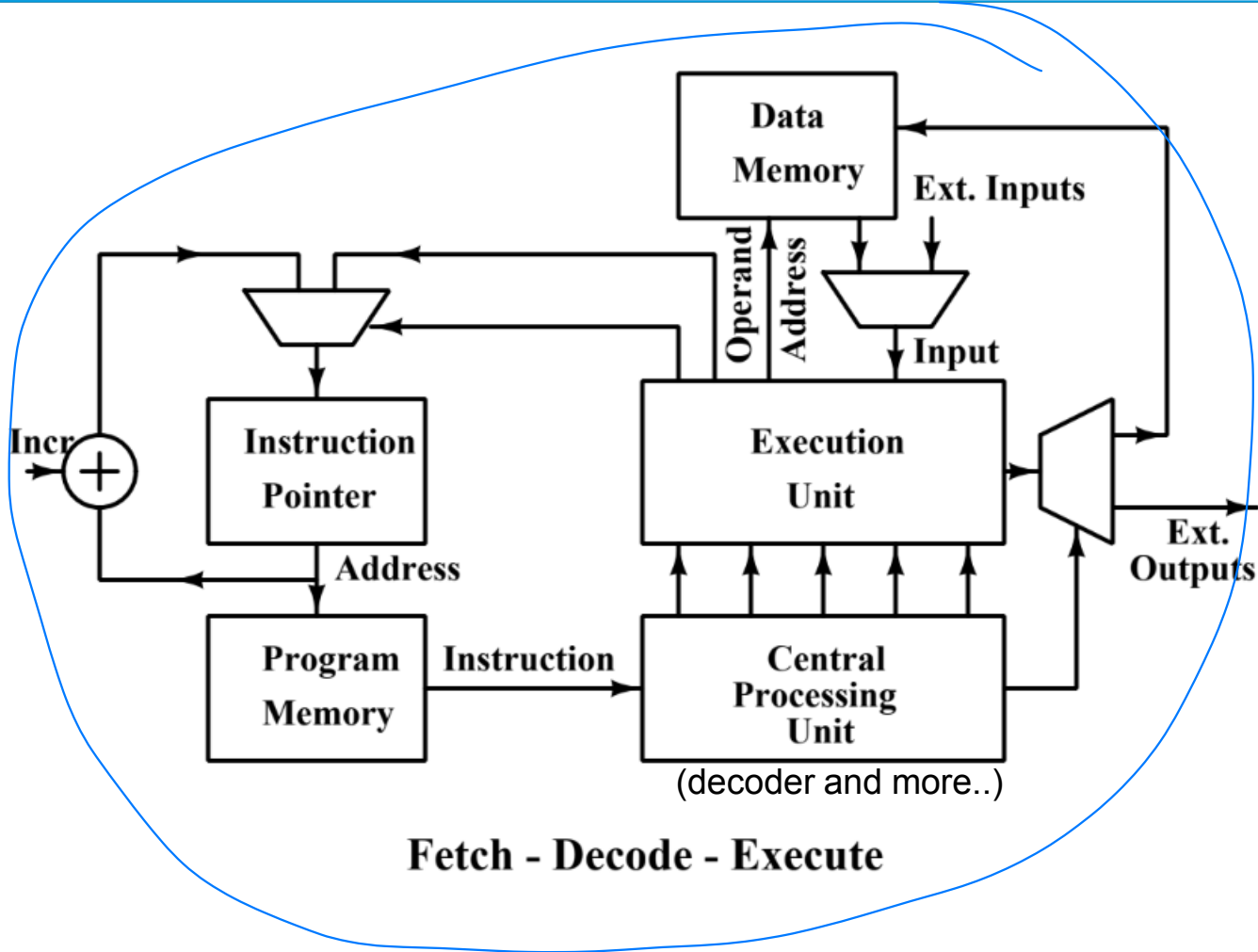


Critical Timing Insight

METASTABILITY



Inside a Processor



Inside ATmega 328P
Microcontroller IC
8-bit Processor

Summary of the course

Resistor heatup kare kyun, Cap DC se dare kyun,
Inductor ko kyun slow pasand hai

Pehle DC response kyun, Phir transient response kyun
Frequency response kyun karna hai

Waveforms kabhi katti kyun, Aur kabhi ye shift hoti kyun
Diode ne kyun machaya hai

Signal amplify hota kyun, Aur filter bhi hota kyun,
Opamp se kya kya mumkin hai

Socha hai.....Yeh tumne kya kabhi
Socha hai.....Ki hai yeh kya sabhi
Socha hai.....Socha nahi toh socho abhi

Summary of the course

Transistor switch kare kyun, zero-one mein badle kyun
Digital isse kyun banta hai

NAND gate hai Universal kyun, XOR hai special kyun
Expression minimize kyun karna hai

Latch level sensitive kyun, flip-flop edge sensitive kyun,
Flip flop ke ander bhi latch kyun hai

Socha hai.....Yeh tumne kya kabhi
Socha hai.....Ki hai yeh kya sabhi
Socha hai.....Socha nahi toh socho abhi

Counter pulse pe badhta kyun, register data rakhta kyun
Flip flop store kyun karta hai
States clock pe badalte kyun, Inputs calculate karte kyun
Processors mein clock kyun hai

Socha hai.....Yeh tumne kya kabhi
Socha hai.....Ki hai yeh kya sabhi
Socha hai.....Socha nahi toh socho
abhi

